BODHI MULE HUNTER AI

Real-time mule account & suspicious transaction detection

Nine layers. Three complementary model views. One calibrated score an investigator can act on — and a containment layer that refuses to act without corroboration.

0.9966	0.9707	99.99%	75×	0.054 ms
FUSED ROC-AUC	FUSED PR-AUC	FALSE POSITIVES CUT	PRECISION UPLIFT	INLINE DECISION P50

Akshay Tiwari 0246CS241037

Palak Vishwakarma 0246AL241124

Archi Singh Rajput 0246CS240174

Kartik Jain 0246AL241094

Team BODHI · CyberShield Hackathon 2026 - Problem Statement 2 · In association with Bank of India and IIT Hyderabad

THE PROBLEM

A mule account is a normal account

Real KYC. Real customer. Payments clear. Nothing about the account is anomalous — only the shape of the money moving through it, and the company it keeps.

HOW A MULE RING ACTUALLY WORKS

1. **Victims** — 40 defrauded people push money into one collector, in 90 minutes
2. **Layering** — A → B → C → D within minutes, each hop keeping 2–10%
3. **Structuring** — Split into transfers parked just under the reporting limit
4. **Cash-out** — Drained via ATM/AePS at 03:12 — out of the banking system

WHY TODAY'S RULE ENGINES FAIL

They score accounts in isolation, so they cannot express “this account's counterparties are themselves suspicious” or “these forty credits arrived in ninety minutes”. Lacking the vocabulary, they compensate with sensitivity.

9,647	1.33%
ALERTS RAISED	PRECISION

99 of every 100 investigations are wasted. That is the number we set out to fix.

THE CORE INSIGHT

Aggregates destroy the signal that matters

The same monthly totals describe a shopkeeper and a mule. Only the ordering separates them — and no aggregate preserves order.

SHOPKEEPER

“Received ₹4 lakh from 38 payers over the month.”

Spread across weeks. Money rests. Counterparties repeat.

MULE

“Received ₹4 lakh from 38 payers between 14:10 and 15:40, then withdrew ₹3.8 lakh in nineteen ATM visits beginning at 03:12.”

Identical aggregates. Completely different sequence.

So the ensemble is not three flavours of the same tabular model. It is a tabular model, a graph neural network and a temporal model — each seeing something the other two structurally cannot.

ARCHITECTURE

Nine layers, each owned by a named agent

Declared input/output contracts, so the provenance of every decision is recoverable from the audit trail.

INGEST & ENRICH

L1	**Transaction ingestion**	UPI · IMPS · NEFT · RTGS · AePS · ATM · card + NCRP tickets + RBI feeds
L2	**Feature engineering**	60 features: velocity, fan-in/out, dormancy, burst, structuring, device reuse
L3	**Graph construction**	accounts × devices × IPs × VPAs, shared-identifier cliques capped

DETECT

L4	**XGBoost screening**	tabular behaviour → exact TreeSHAP attributions
L5	**GraphSAGE**	network structure → rings, device farms, multi-hop layering
L6	**Temporal graph network**	event ordering → smurfing, dormant bursts, rapid routing

DECIDE & ACT

L7	**Risk fusion**	monotone non-negative blend, isotonic calibrated 0–100
L8	**Explainability**	TreeSHAP + GNNExplainer + plain-English narrative
L9	**Alerting & kill-switch**	proportionate, corroborated, reversible, audited

BODHI SHIELD (APK static triage) feeds extracted UPI handles and beneficiary accounts into Layer 3 — priming the graph before the first victim installs the trojan

LAYERS 4–6

Three views that cannot see each other's evidence

LAYER 4

XGBoost

SEES

Aggregates per account

CATCHES

Volume, ratios, cash-out share, structuring

BLIND TO

Who the counterparties are

AUC 0.9946

LAYER 5

GraphSAGE

SEES

Two-hop neighbourhood

CATCHES

Rings, device farms, multi-hop layering

BLIND TO

When things happened

AUC 0.9962

LAYER 6

Temporal GNN

SEES

Last 64 events, in order

CATCHES

Smurfing, dormant bursts, rapid routing

BLIND TO

Anything outside the account

AUC 0.9628

Both neural layers are implemented in **NumPy with hand-derived gradients** — no PyTorch, no CUDA, no 2 GB of wheels. Every analytic gradient is verified against central finite differences in the test suite, so “hand-rolled” does not mean “unverified”.

LAYER 7

Fusion: two properties that are not negotiable

MONOTONICITY

Blend weights are constrained non-negative, so more evidence can never lower a score. An unconstrained stacker on a small fold reliably learns “higher temporal score means safer” — indefensible to an investigator, unshippable past a regulator.

CALIBRATION

The score drives an automated kill-switch at 85, so it has to mean what it says. Isotonic regression maps the blend onto observed frequencies.

0.00125
BRIER SCORE

0.0047
EXPECTED CALIB. ERROR

FOUR FOLDS, NOT THREE

`train` fits the base models

`val` early-stops them

`fuse` fits the Layer-7 blend

`test` touched exactly once

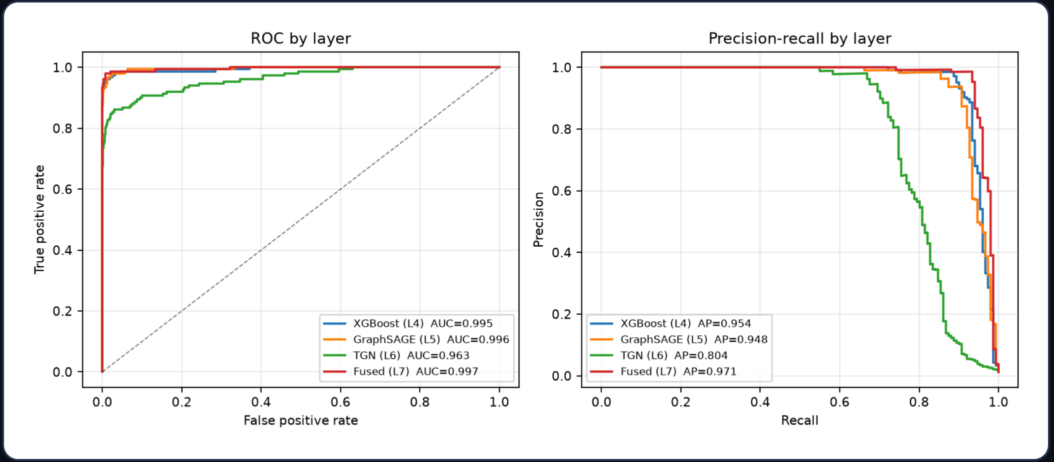
A stacker fitted on the fold that early-stopped its own inputs inherits their optimism — and came out worse than the best single layer.

RESULTS

Every layer, on held-out data

12,000 accounts · 834,738 transactions · 151 mules (1.26%) · 21 rings

LAYER	ROC-AUC	PR-AUC	P@1%	R@1%
XGBoost (L4)	0.9946	0.9540	0.983	0.781
GraphSAGE (L5)	0.9962	0.9481	0.983	0.781
TGN (L6)	0.9628	0.8036	0.892	0.709
Intelligence	0.6883	0.3774	0.475	0.378
Fused (L7)	**0.9966**	**0.9707**	**0.992**	**0.788**



The fused model beats every individual view on both ranking metrics — which is the complementarity claim, measured.

100%
OUT-OF-TIME RECALL

64%
NEVER IN ANY NCRP TICKET

0.113 ms
INLINE DECISION P99

THE HEADLINE

Versus the rule engine it replaces

Compared at identical recall — because a system that alerts on everything can always claim to catch more.

RULE ENGINE · 8 SCENARIOS

Alerts raised	**9,647**
True positives	**128**
False positives	**9,519**
Precision	**1.33%**

→

BODHI · SAME RECALL

Alerts raised	**129**
True positives	**128**
False positives	**1**
Precision	**99.22%**

99.99% of false positives eliminated at unchanged recall — a 75× precision uplift. The analyst clears the same true positives while opening 129 cases instead of 9,647.

WHY THAT NUMBER IS CREDIBLE

We planted the accounts designed to break it

Any model scores ~1.0 AUC on naive synthetic fraud data. So the simulated bank is stocked with legitimate accounts that are structurally indistinguishable from mules.

LEGITIMATE POPULATION	COUNT	FALSE-POSITIVE RATE
Business Correspondent (Bank Mitra) agents	302	**0.33%**
Community collectors (chit fund, tuition)	107	**2.80%**
Small businesses with heavy fan-in	168	**0.00%**
Genuine dormancy reactivation	168	**3.57%**
Ordinary retail	10,504	**0.77%**

THE HARDEST DECOY

A Business Correspondent legitimately operates 8–22 village accounts from one handheld and dispenses AePS cash all day.

That is the exact fingerprint of a device farm doing rapid cash-out — and the system does not fall for it.

OTHER GUARDS AGAINST FLATTERING OURSELVES

Four disjoint folds
test touched exactly once

Cross-fitted seed model
no label leaks into a neighbour's average

Intelligence quarantined
NCRP tickets never reach the supervised layers

Point-in-time correctness
tested against a truncated ledger

WHERE IT IS WEAK

Recall is not uniform, and averaging hides that

A deployment team needs to know which typologies still need human-designed scenarios alongside the model.

TYPOLOGY / ROLE	RECALL	DETECTED
SMURFING	88.9%	16/18
role: CASHOUT	92.1%	35/38
FAN IN AGGREGATION	93.3%	14/15
LAYERING CHAIN	98.3%	58/59
role: COLLECTOR	98.3%	59/60
APK HARVEST	100.0%	15/15
DEVICE FARM	100.0%	31/31
DORMANT BURST	100.0%	13/13
role: RELAY	100.0%	53/53

WHY STRUCTURING IS THE HARDEST

A single sub-threshold transfer is, by construction, indistinguishable from a genuine near-threshold property or jewellery payment. Detection depends on an aggregate pattern that only becomes visible once enough transfers have accumulated.

AND CASH-OUT ACCOUNTS

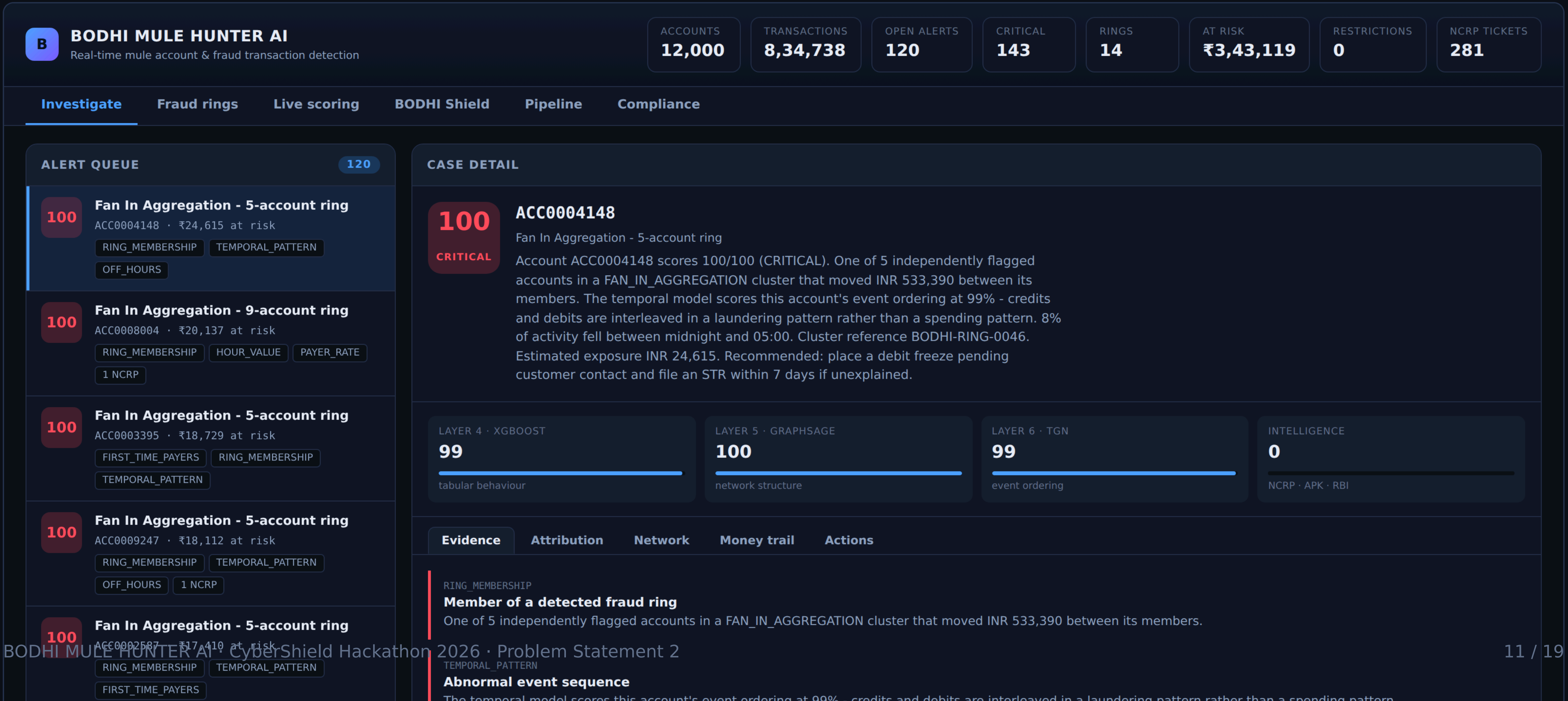
Short, thin histories — they receive once and drain. There is simply less evidence to work with.

We report these rather than averaging them away. A prototype that hides its blind spots is not useful to the team that has to deploy it.

LIVE DEMO

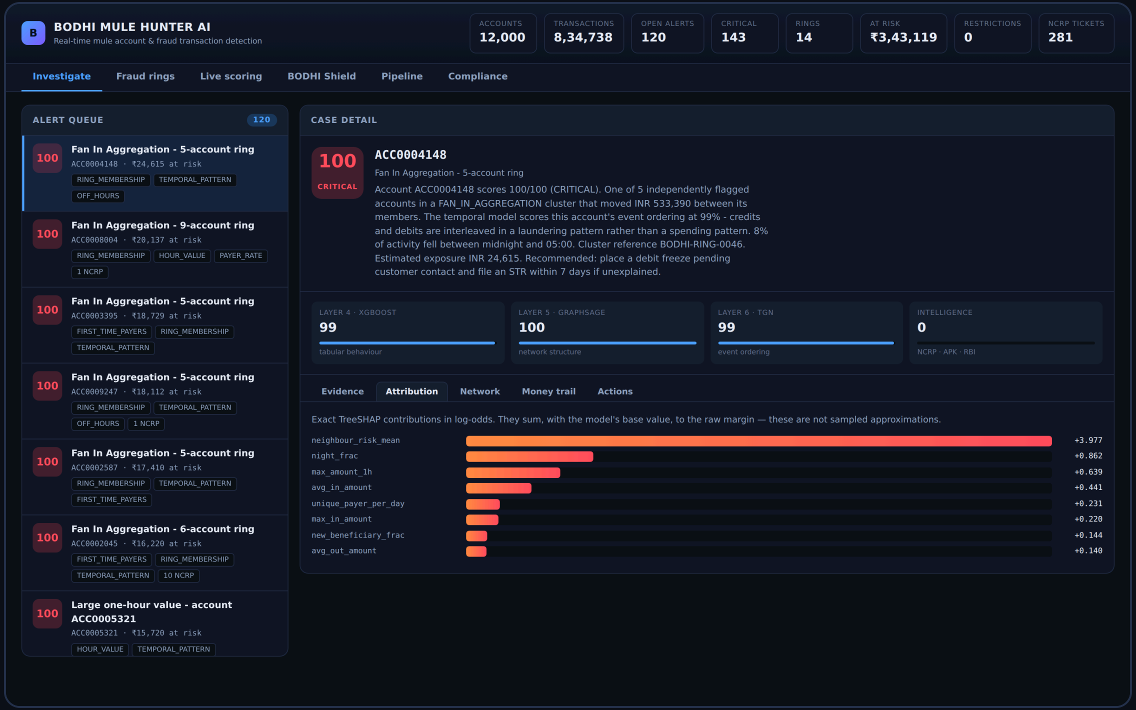
The investigator console

Four layer scores shown separately, evidence written in the vocabulary of an AML desk, cluster and exposure on every card.

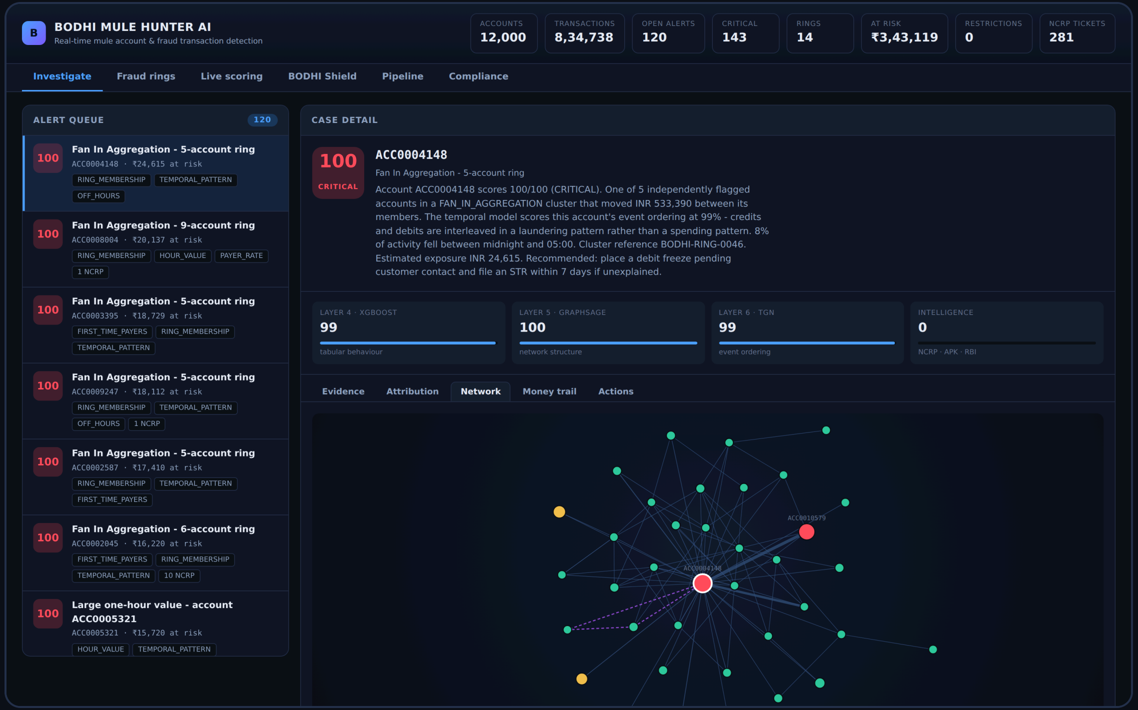


LAYER 8

Explanations specific enough to file



Exact TreeSHAP — attributions sum to the model's margin, not an approximation of it. Rendered as sentences, not feature names.

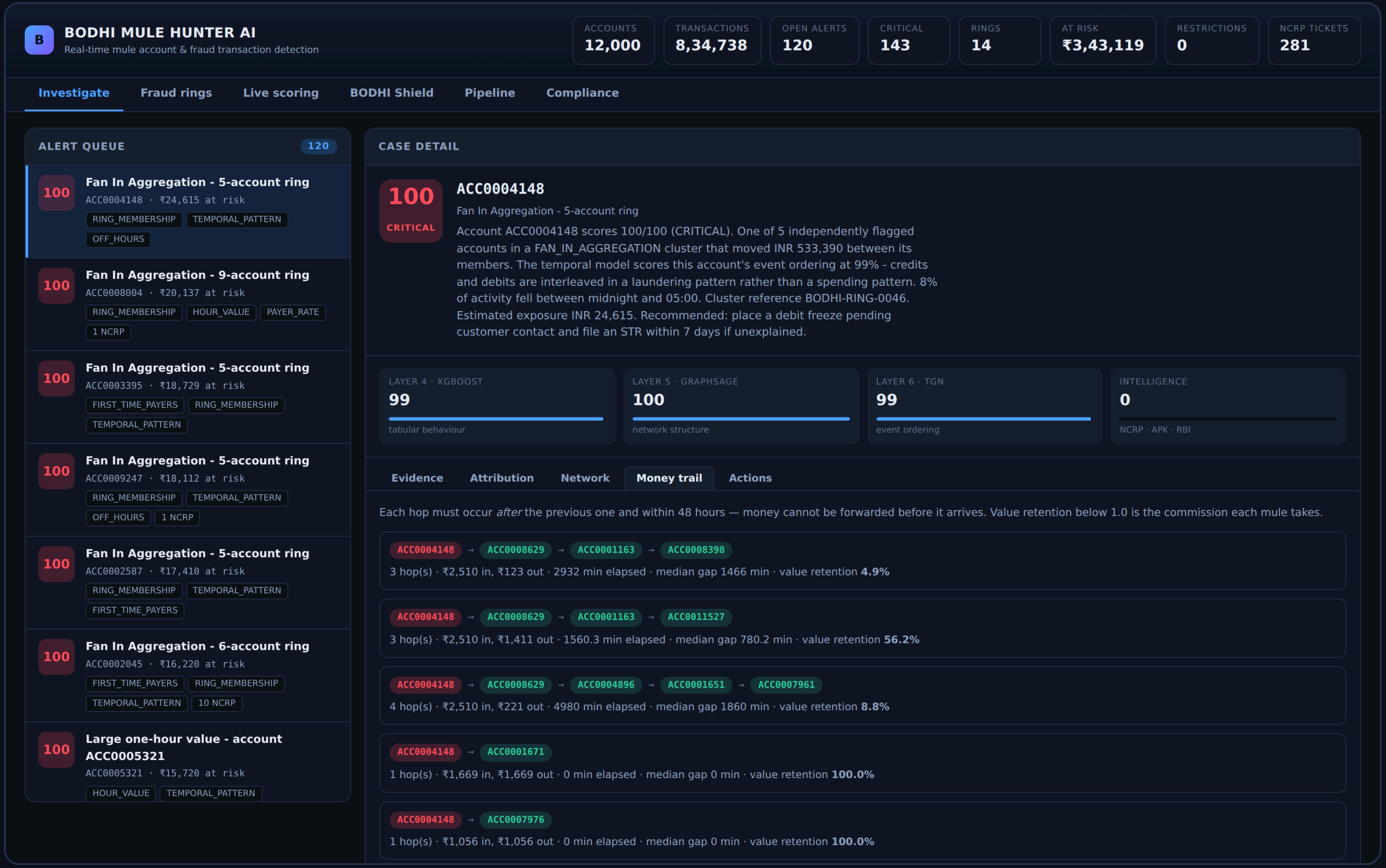


GNNExplainer answers which *relationships* drove the score. Dashed edges are shared-device links, invisible to any tabular model.

FOLLOWING THE MONEY

Time-respecting multi-hop trails

Every hop must occur after the previous one — money cannot be forwarded before it arrives. Value retention exposes each mule's commission.



THE TELL

A laundering chain leaks 2–10% at every hop as the handler's cut.

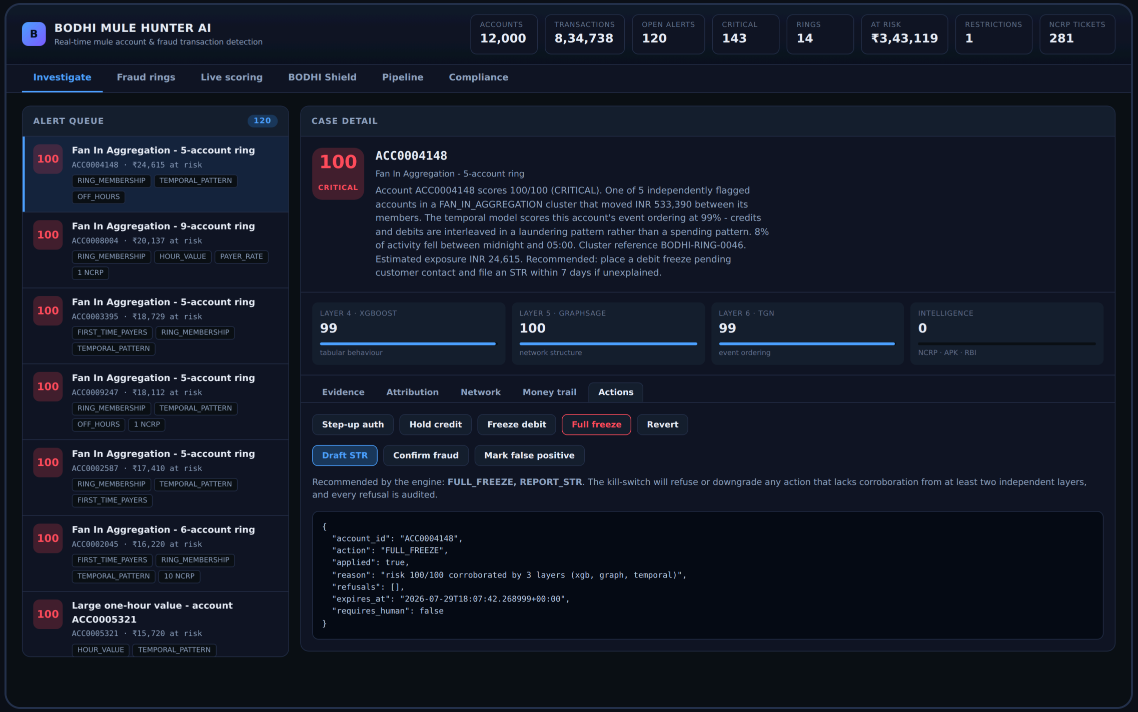
Legitimate payment chains simply do not have that shape — which is why value retention is shown on every path.

LAYER 9

Containment that can refuse

A wrongly frozen account is somebody unable to pay for medicine. No AUC improvement justifies being casual about that.

Proportionality	below 85 the automation cannot freeze at all
Corroboration	a full freeze needs ≥2 independent layers agreeing
Reversibility	every action carries a TTL and can be reverted
Rate limiting	bounded automated freezes per hour caps blast radius
Protected accounts	salary, pension and benefit accounts go to a human

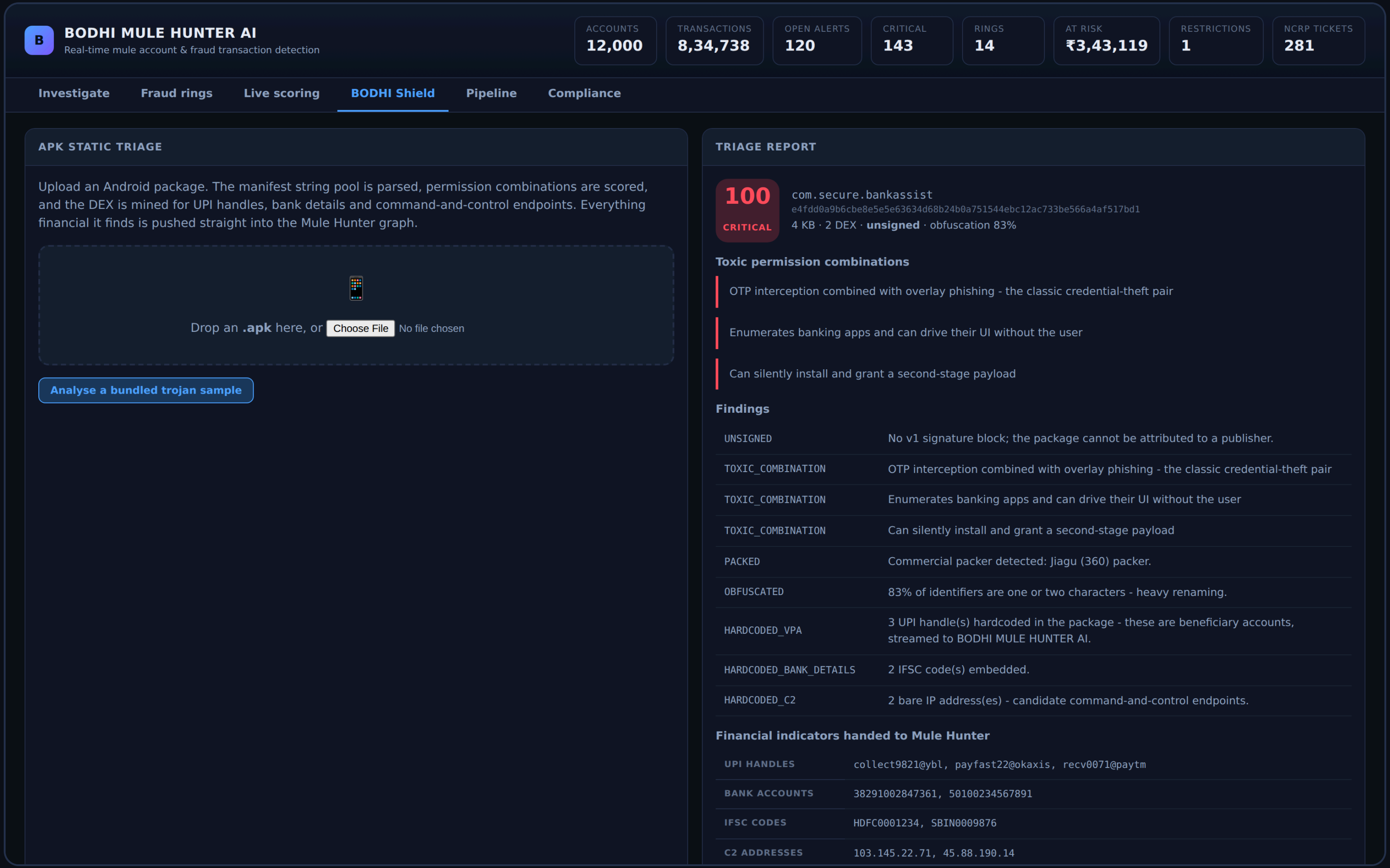


When a constraint fires, the action is **downgraded** — never escalated — and flagged for human review. Every decision, including every refusal, is written to a hash-chained audit log whose head hash can be externally anchored.

ECOSYSTEM SYNERGY

BODHI SHIELD → BODHI MULE HUNTER

Malware analysis priming fraud controls before the first victim installs the app.



1. **Parse manifest**
   binary AndroidManifest.xml string pool, to spec
2. **Score combinations**
   READ_SMS alone is a messaging app; + accessibility + overlay is a trojan
3. **Mine the DEX**
   UPI handles, IFSC codes, accounts, C2 endpoints
4. **Hand off**
   identifiers become weighted nodes in the fraud graph

DEPLOYMENT

How it fits a real bank

The split between scheduled graph work and the inline path is what makes a UPI-latency decision possible at all.

Core banking / switch
Kafka event stream

→

L1–L2 streaming features
feature store

→

L3/L5/L6 batch (scheduled)
standing risk cache

→

Inline decision < 0.1 ms
ALLOW · REVIEW · HOLD · BLOCK

MEASURED PERFORMANCE

Simulate the bank	7 s
Feature engineering	~4 s
Graph construction	~3 s
Full training, all layers	~123 s
Full population re-score	~38 s
Inline decision (p50)	0.054 ms

WHY THIS SPLIT

The graph and temporal layers run on a schedule and cache a standing risk per account. The authorisation path combines those cached scores with the transaction's own attributes and never traverses the graph — which is the only way a decision is feasible at payment latencies.

THEIR DATASET

Built on the schema they published

3,924 declared columns, one row per alert. The names are machine-generated from a grammar, so we parse them instead of treating them as opaque.

FOUR COLUMNS LEAK THE LABEL

FRAUD_SUSPECTED · FALSE_POSITIVE · OTHER_RESOLUTION · UNATTENDED are resolution-status flags — how an analyst *closed* the alert. An open alert has none of them.

PR-AUC, quarantined (deployable) **0.177**

PR-AUC, resolution columns admitted **0.972**

FOUR STRATEGIES, SELECTION INSIDE EVERY FOLD

STRATEGY	FEATS	ROC-AUC	PR-AUC
Bank's 18 finalised	18	0.712	0.177
Bank + engineered	2,028	0.651	0.163
Automatic top-*k*	131	0.622	0.137
Every column	5,926	0.618	0.112

The bank's eighteen expert-chosen columns beat all 5,926. With 186 positives against thousands of predictors, that is what theory predicts.

0.7046 UNTOUCHED HOLDOUT ROC-AUC

0.7116 CROSS-VALIDATED ESTIMATE

5.5× PR-AUC INFLATION IF LEAKED

3,924 COLUMNS PARSED BY GRAMMAR

Measured on a stand-in table with their exact schema — their data was not released when this was built. It proves the pipeline runs and does not flatter itself. It is not model performance, and we will not present it as such.

LIMITATIONS

Stated plainly, because hiding them helps nobody

Simulated data	Demonstrates the architecture beats a rule engine on data with realistic decoys. Not a forecast of production performance.
Static APK analysis only	The dynamic sandbox with runtime hooking needs an instrumented Android image; it cannot ship self-contained.
Graph layers are not real-time	~38 s full re-score. An account whose neighbourhood changed since the last batch is scored on slightly stale structure.
Single-institution view	Rings routing through several banks are only partly visible — a data-sharing problem, not a modelling one.
Cold start	The strongest feature is neighbourhood risk propagated from confirmed cases; with zero known mules performance drops materially.
Disparate impact unevaluated	Minimum-KYC and shared-device features are predictive but correlate with lower-income households. Alert-rate parity must be measured before go-live.

Mule detection fails today not because the signal is absent

— but because single-account rule engines cannot express it.

Three complementary views, fused under a monotonicity constraint into a calibrated score, cut false positives by 99.99% at unchanged recall — with explanations specific enough to file and containment cautious enough to automate.

113	**~17.2k**	**9**	**0**
TESTS PASSING	LINES OF CODE	LAYERS, ALL RUNNING	EXTERNAL ML FRAMEWORKS

Akshay Tiwari
0246CS241037

Palak Vishwakarma
0246AL241124

Archi Singh Rajput
0246CS240174

Kartik Jain
0246AL241094

`make setup && make all && make serve` · `github.com/deepakvish001/BOI-Hackathon-Prototype-`